

AGENTPAY GUARD

Hybrid Blockchain Kill Switch & Policy-Enforced Wallet for Autonomous AI Agents

Beginner-Friendly Hackathon Blueprint — Smart Contract + Backend + Dashboard

Core idea: Keep fast policy checks in a normal backend, but put the final financial authority on-chain. The AI receives only restricted permission; the owner retains an independent on-chain freeze/revoke path.

1. Executive Summary

This version upgrades AgentPay Guard into a hybrid system. A backend policy engine provides easy-to-build controls and monitoring, while a smart-contract wallet provides independent, tamper-resistant enforcement over actual blockchain funds. The autonomous agent never owns the wallet's master authority.

Layer	Role
AI Agent	Requests transactions; cannot edit policy or unfreeze itself.
Policy API	Checks business rules, merchant mapping, limits, rate controls, and prepares transactions.
Smart Contract Wallet	Final on-chain enforcement: frozen state, allowlist, limits, agent authorization.
Owner Wallet	Administrative authority: freeze, revoke agent, update limits/allowlist, recover funds.
Dashboard	Human control plane and transaction visibility.
Blockchain/Testnet	Stores wallet state and executes/rejects transactions.

2. Why Use a Hybrid Design?

Do not move every feature onto blockchain. For a beginner team, blockchain should protect the security-critical controls while normal software handles UX, agent orchestration, analytics, and logs.

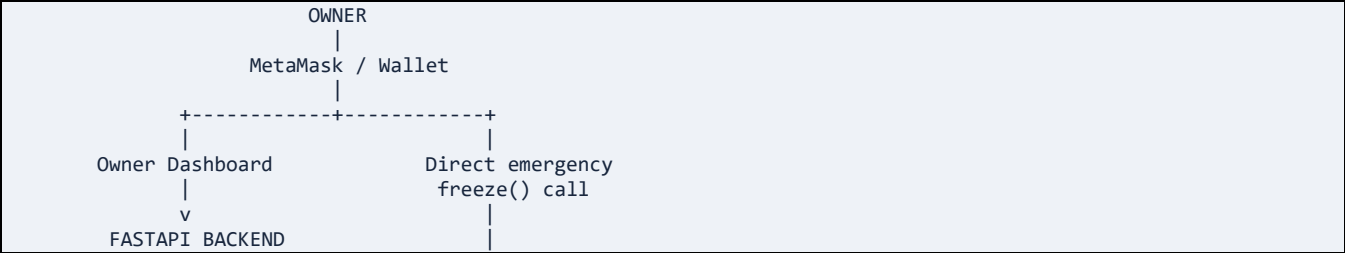
On-chain	Off-chain
Wallet balance / asset custody	LLM reasoning
Frozen/unfrozen state	Dashboard
Agent authorization	Human-readable merchant names
Per-transaction / cumulative limit	Risk scoring / anomaly detection
Approved destination addresses	Detailed audit analytics
Transaction execution	Notifications and agent workflow

3. Security Model

Security invariant: The agent must not possess the owner private key or any key capable of changing wallet policy.

- Owner key: full administrative authority over the smart wallet.
- Agent key: restricted execution authority only.
- Backend: can assist and relay transactions, but cannot bypass smart-contract rules.
- Smart contract: independently rejects disallowed transactions even if the agent/backend is compromised.
- Emergency freeze: owner calls freeze() directly on-chain; agent cooperation is unnecessary.

4. Architecture





5. Beginner Tech Stack

Component	Recommended technology	Purpose
Smart contract	Solidity	Policy-enforced wallet contract.
Contract IDE first	Remix	Fastest beginner workflow for compile/deploy/test.
Local contract tooling later	Hardhat	Scripts, tests, deployment automation.
Blockchain	Ethereum-compatible testnet	Use test funds only for hackathon.
Owner wallet	MetaMask	Owner signs admin actions.
Contract interaction	ethers.js or web3.py	Backend/frontend reads and calls contract.
Backend	Python + FastAPI	Agent API, policy layer, logs, relayer logic.
Database	SQLite	Off-chain metadata and readable audit records.
Dashboard	Streamlit (simplest) or React	Owner controls and live activity.
AI agent	Python + optional LLM API	Autonomous transaction requester.

6. Smart Contract MVP

Keep the contract intentionally small. It needs one owner, one authorized agent address, an allowlist, spend limits, freeze state, and an execute function.

```

contract AgentGuardWallet {
    address public owner;
    address public agent;
    bool public frozen;

    uint256 public perTxLimit;
  
```

```

uint256 public periodLimit;
uint256 public spentThisPeriod;

mapping(address => bool) public allowedTargets;

function execute(address target, uint256 amount, bytes calldata data) external;
function freeze() external;
function unfreeze() external;
function setAgent(address newAgent) external;
function revokeAgent() external;
function setAllowedTarget(address target, bool allowed) external;
function setLimits(uint256 perTx, uint256 period) external;
}

```

Inside `execute()`, enforce checks before the external call. For the hackathon, a simple resettable spending period is acceptable; clearly describe it as a demo implementation rather than production accounting.

7. Core On-Chain Checks

```

require(msg.sender == agent, "NOT_AGENT");
require(!frozen, "WALLET_FROZEN");
require(allowedTargets[target], "TARGET_NOT_ALLOWED");
require(amount <= perTxLimit, "PER_TX_LIMIT");
require(spentThisPeriod + amount <= periodLimit, "PERIOD_LIMIT");

// Update accounting before external execution.
// Use safe patterns / reentrancy protection in the real implementation.

spentThisPeriod += amount;
(bool ok, ) = target.call{value: amount}(data);
require(ok, "EXECUTION_FAILED");

```

8. Transaction Workflow

1. Owner deploys the smart wallet and becomes contract owner.
2. Owner authorizes the AI agent's blockchain address.
3. Owner configures per-transaction limit, cumulative limit, and allowlisted destination addresses.
4. Owner deposits testnet funds into the smart wallet.
5. AI decides it needs a payment and sends a request to FastAPI.
6. Backend performs optional off-chain checks and converts a merchant ID into its approved blockchain address.
7. Agent signs/calls `execute(target, amount, data)` using its restricted key.
8. Smart contract independently rechecks authorization, frozen state, allowlist, and limits.
9. If valid, funds move from the smart wallet; otherwise the transaction reverts.
10. Contract emits an event; backend/dashboard reads the result and displays it.
11. At any moment, owner can call `freeze()` or `revokeAgent()`.

9. Owner vs Agent Permissions

Action	Owner	Agent
View wallet/status	Yes	Yes
Request/execute allowed payment	Optional	Yes
Change spending limits	Yes	No
Add/remove allowlisted target	Yes	No
Freeze wallet	Yes	No
Unfreeze wallet	Yes	No
Replace/revoke agent	Yes	No

Recover/withdraw funds	Yes	No
Bypass contract checks	No	No

10. Backend API

Method	Endpoint	Use
POST	/payments/request	Agent asks backend to prepare/validate a payment.
GET	/agents/{id}/status	Read on-chain + off-chain status.
GET	/transactions	Readable transaction/event history.
POST	/admin/freeze	Dashboard asks owner flow to prepare freeze action.
POST	/admin/unfreeze	Prepare owner unfreeze action.
PUT	/admin/policy	Update metadata and prepare on-chain policy transaction.
POST	/admin/allowlist	Map merchant and prepare allowlist update.

Important: owner-only blockchain changes should be signed by the owner's wallet. Do not store the owner's private key in the backend for the hackathon.

11. Database

Table	Example fields
agents	id, name, chain_address, status_cache, created_at
merchants	id, name, chain_address, metadata
payment_requests	request_id, agent_id, merchant_id, amount, decision, tx_hash
contract_events	event_name, tx_hash, block_number, decoded_data, timestamp
audit_events	actor, action, details, timestamp

12. Screens

Screen 1 — Owner Control Dashboard

AGENTPAY GUARD — ON-CHAIN CONTROL

Agent: ProcurementAgent-01
Network: Testnet
Smart Wallet: 0xABCD...1234
Agent Key: 0x9876...4321

Status: ACTIVE
Wallet Balance: 0.50 test ETH
Per Tx Limit: 0.01 ETH
Period Limit: 0.05 ETH
Spent: 0.015 ETH

[EDIT LIMITS] [MANAGE ALLOWLIST]
[REVOKE AGENT] [FREEZE WALLET]

Recent:
0.005 ETH -> Approved Vendor CONFIRMED
0.020 ETH -> Approved Vendor REVERTED: LIMIT
0.003 ETH -> Unknown Address REVERTED: NOT ALLOWED

Screen 2 — Allowlist Manager

APPROVED ON-CHAIN DESTINATIONS

Compute Provider 0x1111...1111 [REMOVE]
API Provider 0x2222...2222 [REMOVE]

New address: [0x.....]
Label: [Vendor name]

[ADD TO ALLOWLIST]

Owner wallet signature required.

Screen 3 — Kill Switch Confirmation

EMERGENCY FREEZE

This will submit freeze() from the OWNER wallet.

After confirmation:

- agent remains online
- wallet funds remain safe
- execute() calls from agent revert
- only owner can unfreeze

[CANCEL] [FREEZE ON-CHAIN]

Screen 4 — Live Agent Console

AGENT RUNNING

Need compute -> request 0.005 ETH
Contract: CONFIRMED

Trying 0.020 ETH
Contract: REVERTED — PER_TX_LIMIT

Trying unknown 0xAAAA..
Contract: REVERTED — TARGET_NOT_ALLOWED

OWNER FREEZES WALLET

Trying valid 0.001 ETH
Contract: REVERTED — WALLET_FROZEN

Agent is still running. Financial authority is gone.

Screen 5 — Transaction Explorer

REQUEST	TARGET	AMOUNT	CHAIN RESULT
#101	Compute	.005 ETH	CONFIRMED
#102	Compute	.020 ETH	REVERTED
#103	Unknown	.003 ETH	REVERTED
#104	Compute	.001 ETH	REVERTED (FROZEN)

[Open tx hash in testnet explorer]

13. Attack Demo Matrix

Scenario	What agent tries	Protection
Overspend	Amount above perTxLimit	Contract reverts.
Split payments	Many valid-sized transfers	Cumulative period limit stops further spend.

Unknown wallet	Send to non-allowlisted address	Contract reverts.
Ignore backend	Agent calls contract directly	Same on-chain checks still apply.
Compromised agent key	Attacker uses agent key	Still restricted by allowlist/limits/freeze.
Change policy	Agent calls setLimits/setAllowedTarget	Owner-only authorization rejects it.
Post-freeze	Agent sends valid payment	Contract reverts WALLET_FROZEN.

14. Kill Switch Reliability

The strongest improvement over the non-blockchain version is that the owner has an independent on-chain control path. Even if the AI process or backend is malfunctioning, the owner wallet can call freeze(). Once that transaction is confirmed, subsequent execute() calls must fail.

```

Owner wallet
|
| freeze()
v
Smart Wallet: frozen = true
|
+-- Agent execute(...) -> REVERT
+-- Agent execute(...) -> REVERT
+-- Agent execute(...) -> REVERT

```

15. In-Flight Revocation: Be Precise

A transaction that is already finalized on-chain cannot simply be cancelled. For the bonus, implement a two-step queued transaction for higher-value payments: request/queue first, execute after a delay. The owner can freeze or cancel during the delay.

```

Agent -> queuePayment(target, amount)
      |
      v
    PENDING / TIMELOCK
      |
    delay (e.g. 30 sec)
      |
      v
    executeQueued()
      |
    recheck frozen
      /      \
    active    frozen
      |        |
    execute   cancel/revert

```

For a beginner MVP, this should be a stretch feature. The base kill switch is already strong.

16. Contract Events

```

event PaymentExecuted(address indexed agent, address indexed target, uint256 amount);
event WalletFrozen(address indexed owner);
event WalletUnfrozen(address indexed owner);
event AgentChanged(address indexed oldAgent, address indexed newAgent);
event TargetUpdated(address indexed target, bool allowed);
event LimitsUpdated(uint256 perTxLimit, uint256 periodLimit);

```

Events make the dashboard easy to update and give judges visible proof that policy changes and payments happened on-chain.

17. Folder Structure

```
agentpay-guard/  
+-- contracts/  
|   +-- AgentGuardWallet.sol  
+-- scripts/  
|   +-- deploy.js  
|   +-- seed-policy.js  
+-- test/  
|   +-- AgentGuardWallet.test.js  
+-- backend/  
|   +-- main.py  
|   +-- policy.py  
|   +-- chain.py  
|   +-- database.py  
+-- agent/  
|   +-- agent.py  
|   +-- attack_scenarios.py  
+-- dashboard/  
|   +-- app.py  
+-- data/  
|   +-- agentpay.db  
+-- hardhat.config.js  
+-- requirements.txt  
+-- README.md
```

18. Build Order for Beginners

Stage	Build	Success test
1	Smart contract in Remix	Owner deploys; contract accepts test funds.
2	freeze/unfreeze	Agent payment works before freeze and fails after freeze.
3	allowlist	Approved address works; unknown address reverts.
4	limits	Oversized and cumulative overspend attempts revert.
5	Python agent	Agent automatically sends/requests transactions.
6	FastAPI + SQLite	Readable request and event logs.
7	Streamlit dashboard	Owner sees status and initiates admin actions.
8	Attack demo	All bypass scenarios fail visibly.
9 optional	Queued/timelocked payment	Freeze cancels a pending high-risk payment.

19. Testing Checklist

- Non-owner cannot freeze/unfreeze or modify policy.
- Agent cannot modify allowlist or limits.
- Authorized agent can execute a valid allowed payment.
- Unauthorized address cannot execute.
- Allowed target + excessive amount reverts.
- Unknown target reverts.
- Cumulative spend limit cannot be bypassed by splitting.
- Frozen wallet rejects otherwise-valid payments.
- Unfreeze restores execution only when called by owner.

- Revoked agent key can no longer execute.
- Events are emitted for successful admin/payment actions.

20. Demo Script

12. Show the deployed smart wallet and its testnet balance.
13. Show owner address and separate restricted agent address.
14. Agent sends 0.005 test ETH to an allowlisted vendor: CONFIRMED.
15. Agent tries 0.02 ETH above its limit: REVERTED.
16. Agent tries an unknown address: REVERTED.
17. Agent bypasses FastAPI and calls the smart contract directly: still REVERTED, proving backend independence.
18. Owner presses FREEZE and signs the blockchain transaction.
19. Keep the AI agent running. It attempts a valid payment after freeze: REVERTED WALLET_FROZEN.
20. Show the transaction/event history and, if available, the testnet explorer.
21. Optional: demonstrate queued high-value payment and freeze before execution.

21. Evaluation Metric Mapping

Metric	Evidence
Enforcement layer	Smart contract itself enforces agent authorization, frozen state, allowlist, and limits.
Kill-switch reliability	Owner wallet can freeze directly on-chain, independent of agent cooperation.
Attack resistance	Direct-contract bypass, overspend, split spend, unknown target, and policy-change attacks fail.
In-flight revocation	Optional queued/timelocked payment can be cancelled/frozen before execution.
Real-world plausibility	Uses a real smart-contract wallet model and testnet funds; off-chain services handle practical UX.

22. Important Limitations to State Honestly

- Blockchain confirmation is not literally instantaneous; freeze becomes authoritative according to the network's transaction inclusion/finality behavior.
- A transaction already finalized cannot generally be reversed.
- Allowlisting an address alone does not guarantee the downstream contract cannot forward funds; advanced versions can also restrict callable functions/selectors.
- Private-key management is simplified for a hackathon. Production systems should use hardened key custody, account abstraction/session keys, multisig/recovery, monitoring, and audited contracts.
- Time-based/cumulative accounting in the MVP is simplified; production spending windows require careful contract design.
- Use testnet/test funds for the demo.

23. MVP Recommendation

Best beginner version: Solidity smart wallet + Remix first, Python AI agent, FastAPI, SQLite, Streamlit, MetaMask owner control, and testnet funds.

Do not begin with advanced account abstraction. First prove one simple property end-to-end: the AI is running and has a valid agent key, but the owner-controlled smart wallet can independently block its money.

24. One-Line Pitch

"The AI can decide what it wants to buy, but it never controls the rules of the wallet that pays for it."